

Experiment No. _____

Online Cab Booking System

(SRS Report)

JAMIA MILLIA ISLAMIA
Department of Computer Engineering
Bachelor of Technology

Members:

- | | |
|-----------------|-------------|
| 1. Abdul Nasood | Roll No. 2 |
| 2. Md Nehal | Roll No. 38 |

Table of Contents

1. Introduction.....	1
1.1 Purpose.....	1
1.2 Product Scope.....	1
1.3 Technologies to be Used.....	1
1.4 Definitions, Acronyms and Abbreviations.....	1
2. Overall Description.....	1
2.1 Product Perspective.....	1
2.2 User Classes and Characteristics.....	1
2.3 Operating Environment.....	2
2.4 Software Interfaces.....	2
2.5 Hardware Requirements.....	2
2.6 Functionalities.....	2
2.7 General Constraints.....	2
3. System Features.....	2
3.1 Functional Requirements.....	2
3.2 Technical Issues.....	2
3.3 Non-Functional Requirements.....	2
4. Design.....	3
4.1 Use Case Diagram.....	3
4.2 Activity Diagram.....	4
4.3 Data Flow Diagrams (DFD 0 and DFD 1).....	5
4.4 DFD Level 2: Ride Management.....	6
5. Future Scope.....	7
6. Conclusion.....	7
7. References.....	7

1. Introduction

1.1 Purpose

This document lists the requirements for our Online Cab Booking System. It explains what the system will do, who will use it and how it is designed, so it can be used as a reference while building and testing the project.

1.2 Product Scope

The system lets people book a cab from their phone or laptop. A user enters the pickup and drop location, chooses a vehicle, tracks the cab on a map and pays at the end of the ride. Drivers can accept or reject rides, and the admin takes care of accounts, payments and bookings.

1.3 Technologies to be Used

Technology	Description
Front-end	HTML, CSS, JavaScript and React.js
Back-end	Node.js (Express) or Python (Django)
Database	PostgreSQL
Maps	Google Maps API for location, routes and live tracking
Payments	Razorpay / UPI, and cash
Notifications	SMS, email and in-app alerts

1.4 Definitions, Acronyms and Abbreviations

Term	Meaning
SRS	Software Requirements Specification
API	Application Programming Interface
DFD	Data Flow Diagram
GPS	Global Positioning System, used for live location
ETA	Estimated Time of Arrival
OTP	One-Time Password, used for verification
User	A person who books a ride
Driver	A verified person who drives the cab

2. Overall Description

2.1 Product Perspective

This is a standalone web app. Instead of waving down a taxi on the road or calling for one, users can book, track and pay for a ride in one place.

2.2 User Classes and Characteristics

- **User:** books or cancels a ride, chooses a vehicle, tracks the cab, pays and gives feedback.
- **Driver:** accepts or rejects rides, follows the navigation and collects cash.
- **Administrator:** manages accounts, transactions and bookings.

2.3 Operating Environment

It runs on any modern browser like Chrome, Firefox, Safari or Edge, on both mobile and desktop. The backend will be hosted on a cloud server such as AWS.

2.4 Software Interfaces

The app uses PostgreSQL as the database, a React front-end that talks to the backend through a REST API, Google Maps for location and a payment gateway for online payments.

2.5 Hardware Requirements

- **Server:** 4-core CPU, 16 GB RAM and 500 GB SSD.
- **Client:** any phone or computer with internet. Drivers also need GPS on their phone.

2.6 Functionalities

- **Register and Login:** users and drivers sign up and log in using OTP verification.
- **Book a Ride:** enter pickup and drop, choose a vehicle, check the fare and confirm.
- **Driver Matching:** the nearest free driver gets the request. If nobody accepts, the user can try again.
- **Tracking:** the user can see the cab live on the map, and the driver gets directions.
- **Payment:** pay online or in cash after the ride and get a receipt.
- **Cancel and Feedback:** a ride can be cancelled, and the user can rate the driver after the trip.

2.7 General Constraints

- User data, location and payment details must be kept safe.
- The system should handle a lot of bookings during peak hours.
- Location features depend on GPS and the maps service.

3. System Features

3.1 Functional Requirements

- Users and drivers should be able to register and log in.
- Users should be able to book, cancel and track a ride.
- The system should send each request to the nearest available driver.
- Drivers should be able to accept or reject a ride.
- The system should calculate the fare and accept online or cash payment.
- The admin should be able to manage accounts, transactions and bookings.

3.2 Technical Issues

PostgreSQL will be used as the database, and the system will run on a Linux-based server.

3.3 Non-Functional Requirements

- **Security:** passwords are stored hashed and all data goes over HTTPS.
- **Performance:** find a driver within 30 seconds and load pages within 3 seconds.
- **Reliability:** the system should stay up 99.9% of the time, with regular backups.
- **Usability:** booking a ride should only take a few taps.
- **Scalability:** more servers can be added easily as the number of users grows.

4. Design

4.1 Use Case Diagram

This diagram shows the three actors (User, Driver and Administrator) and what each of them can do in the app.

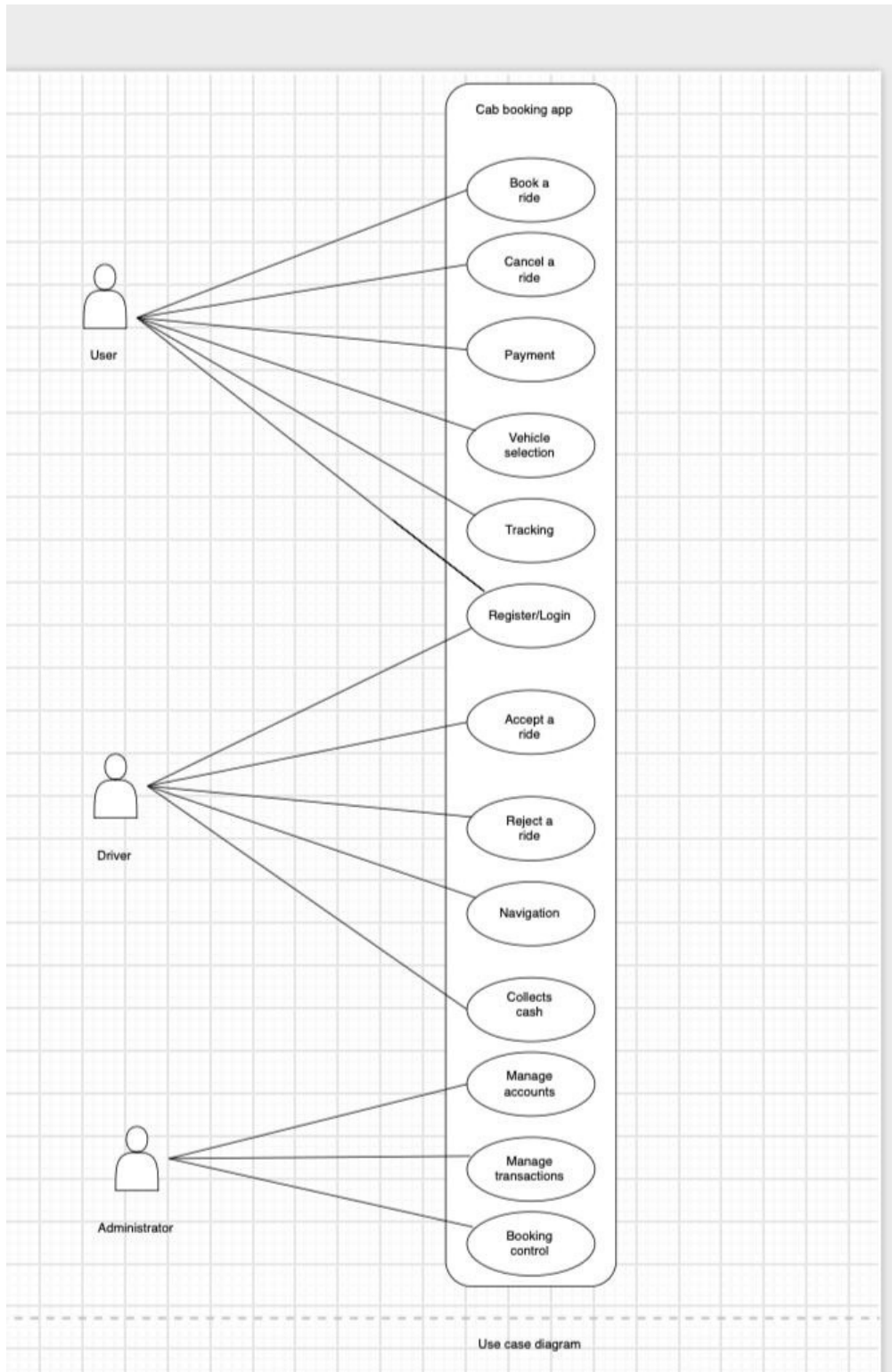


Fig. 4.1 Use Case Diagram

4.2 Activity Diagram

This shows the steps a user goes through to book a ride: logging in, entering the pickup and drop, waiting for a driver, then tracking the ride, paying and giving feedback.

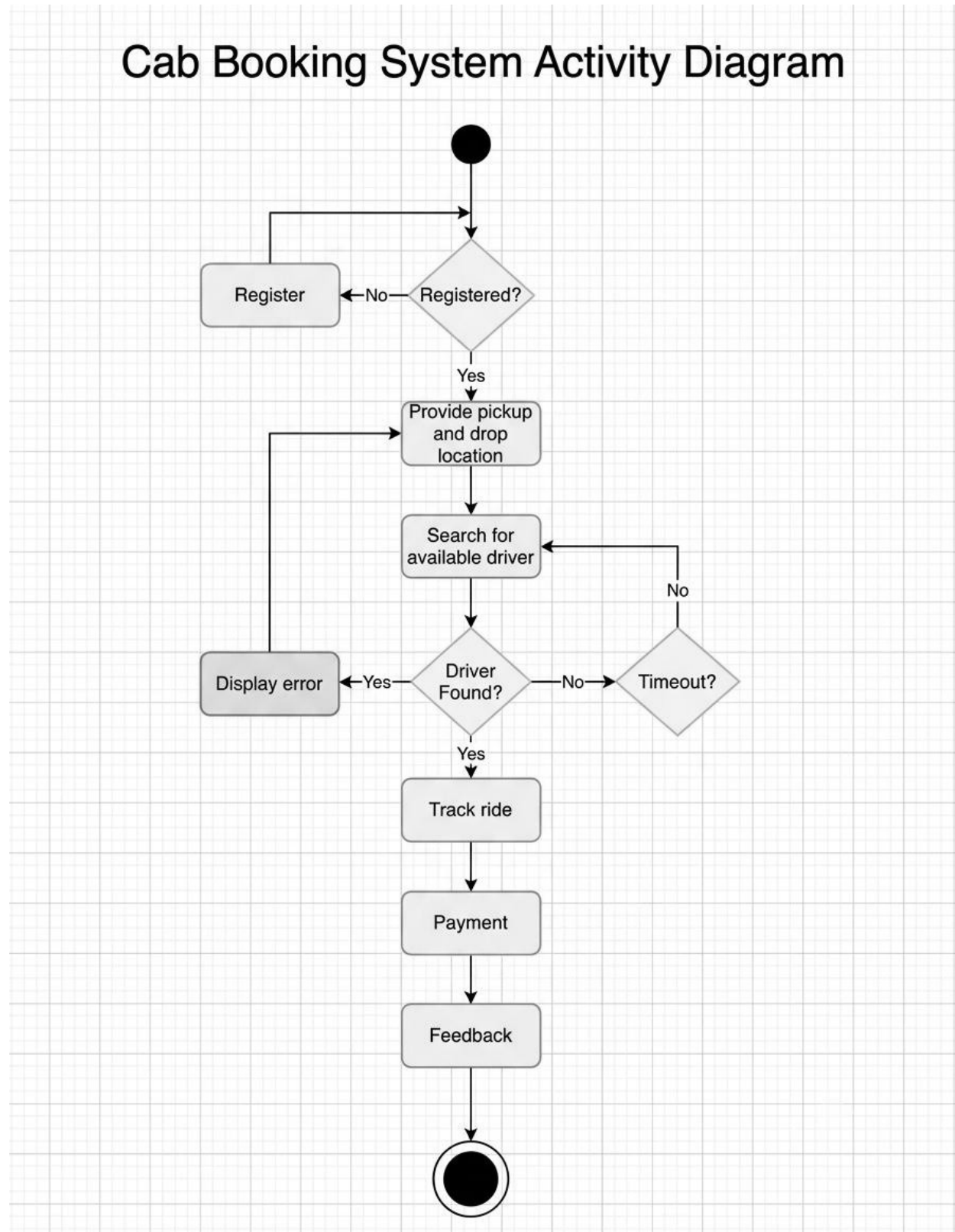


Fig. 4.2 Activity Diagram

4.3 Data Flow Diagrams (DFD 0 and DFD 1)

DFD 0 shows the whole system as one process that talks to the User and the Driver. DFD 1 splits it into registration, booking, ride, driver and payment management, along with the data each one stores.

DFD 0 — Context Level

DFD 0-Context Level

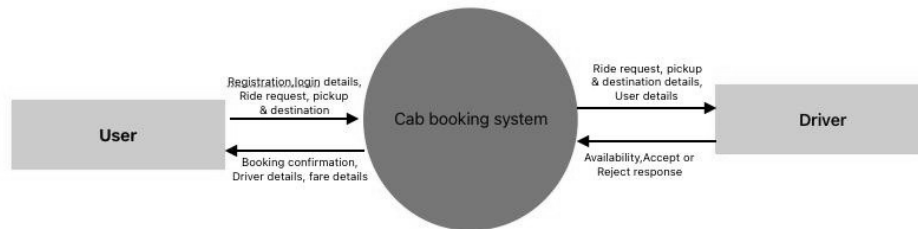


Fig. 4.3 DFD Level 0 (Context Diagram)

DFD 1 — Level 1

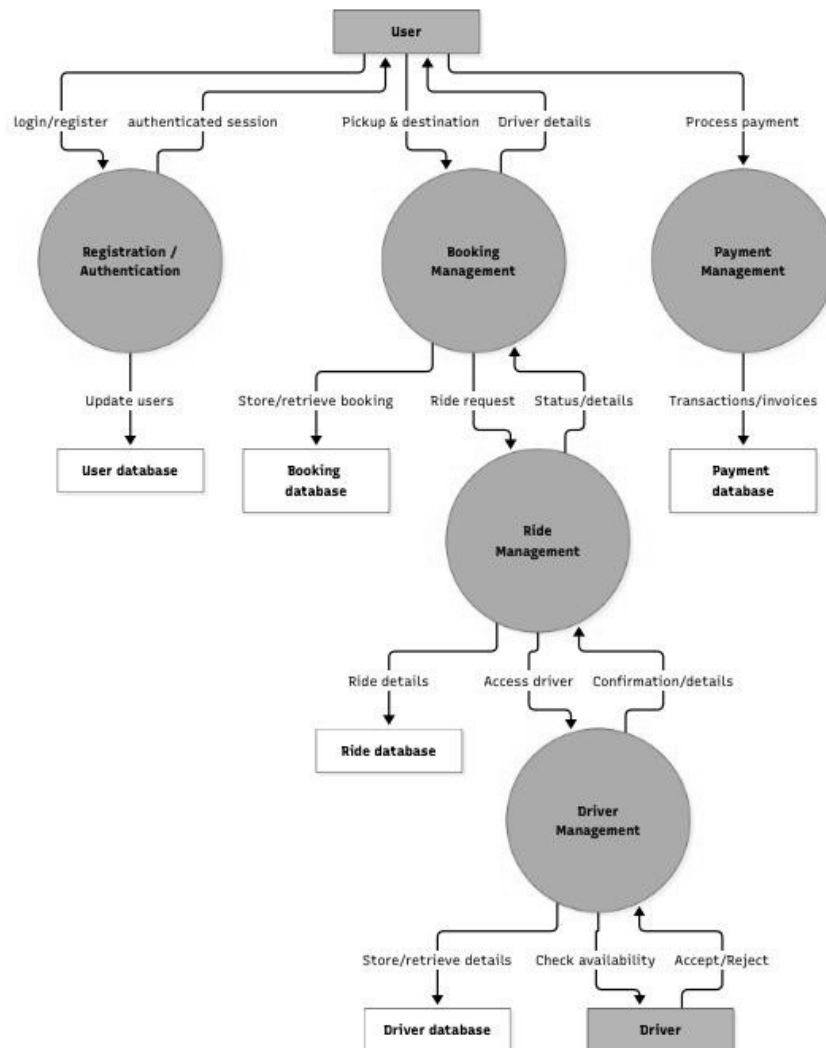


Fig. 4.4 DFD Level 1

4.4 DFD Level 2: Ride Management

DFD 2 zooms into Ride Management from DFD 1 (numbered as process 4). It shows how a ride request becomes an assigned driver, then a started ride, and finally a completed ride with a fare.

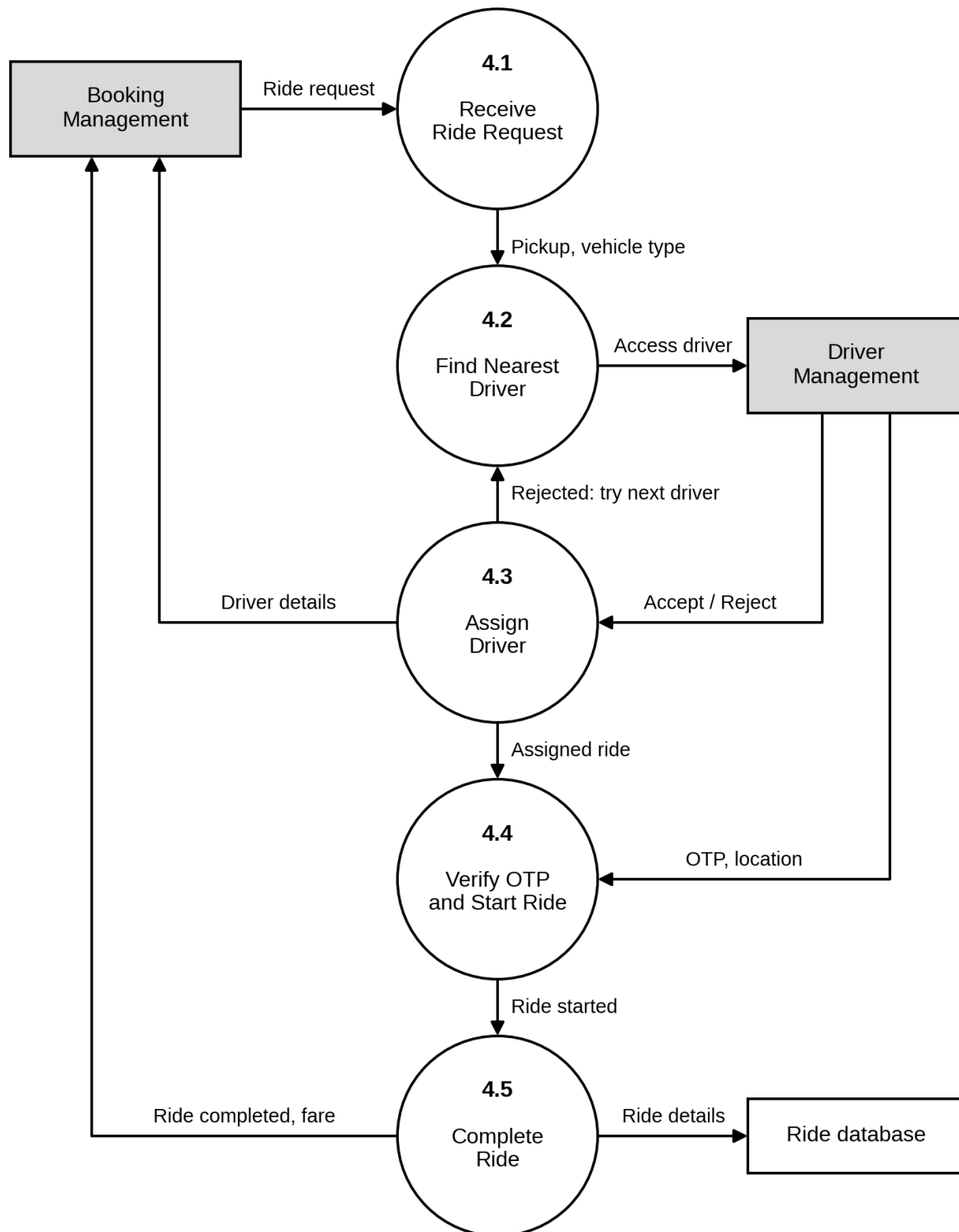


Fig. 4.5 DFD Level 2 (Ride Management)

5. Future Scope

Some features we can add later:

- Ride sharing, so people going the same way can split the fare.
- Separate Android and iOS apps.
- Electric cabs, with nearby charging points shown to drivers.
- Safety features like an SOS button and live trip sharing.
- Using past ride data to predict busy areas and send drivers there early.

6. Conclusion

This SRS gives a clear picture of what the cab booking system needs to do and how it should work. Together with the use case, activity and data flow diagrams, it will guide us while designing, building and testing the project, so that the final app is simple, safe and reliable.

7. References

- Roger S. Pressman, Software Engineering: A Practitioner's Approach
- www.geeksforgeeks.org
- developers.google.com/maps
- IEEE Xplore and Google Scholar